ОТЧЁТ ПО ЛАБОРАТОРНОЙ РАБОТЕ №3

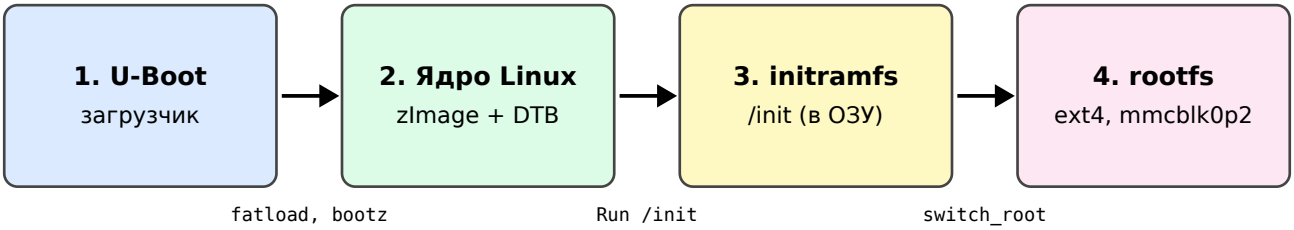
Тема: Сборка компонентов встроенной системы для архитектуры ARM

Целевая платформа: QEMU, плата **vexpress-a9** (ARM Cortex-A9)

Выполнил: Реджеп

1. Цель работы и схема загрузки

Цель работы — собрать из исходных кодов все компоненты минимальной встроенной Linux-системы для ARM и запустить их в QEMU, обеспечив требуемую последовательность загрузки: **загрузчик → ядро → initramfs → rootfs**.



Загрузчик → ядро → initramfs → rootfs (корень сменяется командой switch_root)

Ключевая особенность: initramfs и rootfs — **два разных объекта**. Initramfs (архив, упакованный в uInitrd) ядро распаковывает в оперативную память и запускает из него **/init**. Скрипт /init находит на SD-карте раздел с настоящей корневой файловой системой (ext4), монтирует его и командой **switch_root** передаёт ему управление. После этого корнем системы является раздел SD-карты /dev/mmcblk0p2, а initramfs из памяти удалён.

Состав образа SD-карты (sd.img, 128 МиБ, таблица разделов MBR):

Раздел	Файловая система	Размер	Содержимое
mmcblk0p1	FAT	32 МиБ	zImage, vexpress-v2p-ca9.dtb, uInitrd (читает U-Boot)
mmcblk0p2	ext4	95 МиБ	настоящий rootfs: BusyBox, /etc/inittab, /etc/init.d/rcS, /etc/rootfs-marker

2. Часть 1. Настройка среды разработки

Установлены пакеты **git** и **qemu-system-arm**, а также GNU кросс-компилятор **arm-linux-gnueabi-gcc** и утилита **mkimage** (пакет u-boot-tools), которая нужна для упаковки initramfs в формат U-Boot. Работа выполнялась в WSL (Ubuntu). Версии инструментов показаны на рисунке 1.

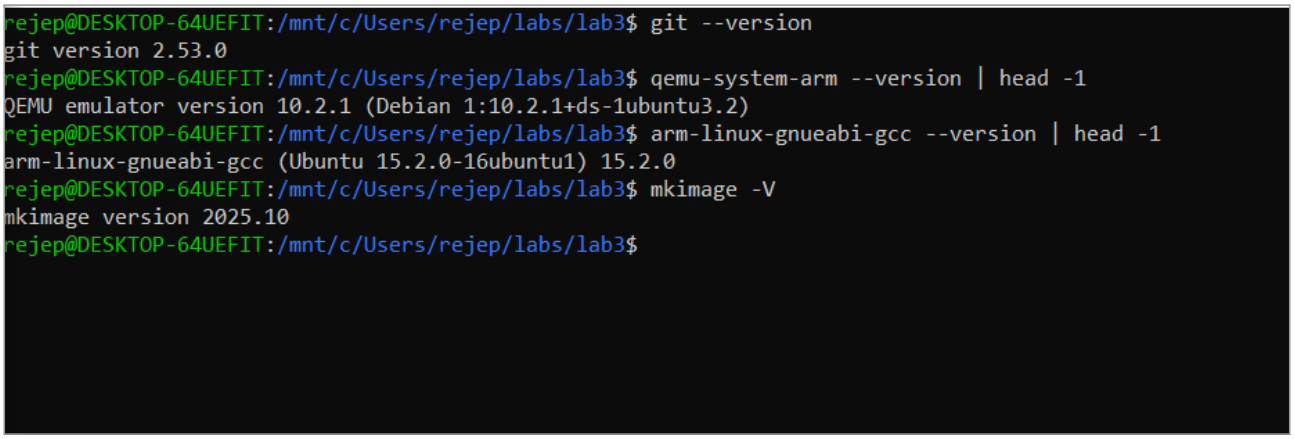


Рисунок 1 — Версии git, QEMU, кросс-компилятора и mkimage

Загружены исходные коды: стабильное ядро **Linux 7.2.9**, загрузчик первого этапа **U-Boot 2026.10** и набор UNIX-утилит **BusyBox 1.37.0** (рисунок 2). Версия ядра видна из первых строк Makefile (VERSION = 7, PATCHLEVEL = 2, SUBLEVEL = 9), для U-Boot показан хэш коммита.

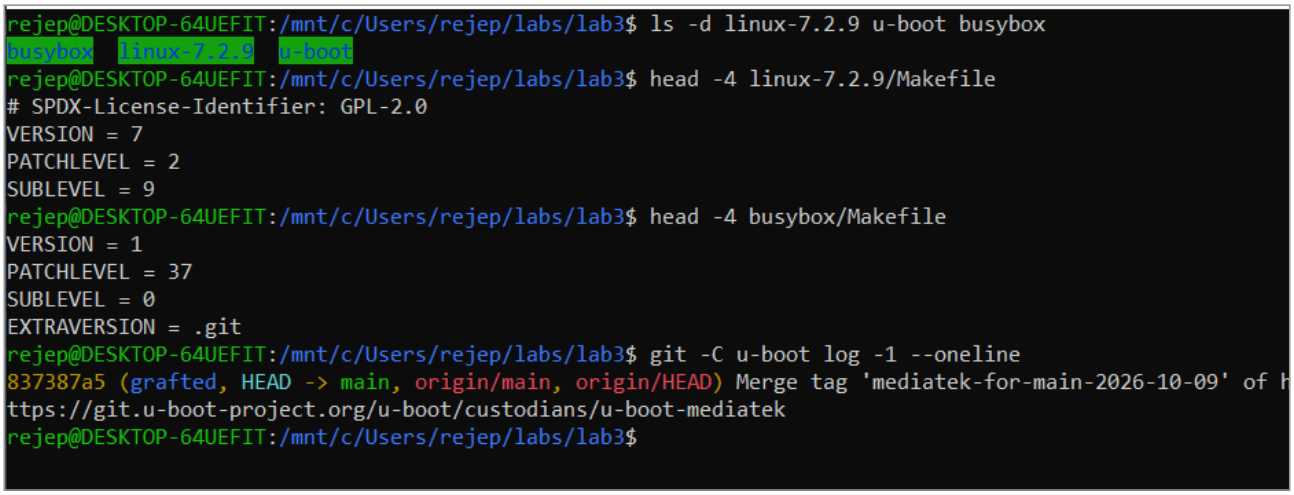


Рисунок 2 — Исходные коды: версии Linux, BusyBox и коммит U-Boot

3. Часть 2. Сборка компонентов

3.1. Загрузчик U-Boot

U-Boot собран кросс-компилятором с конфигурацией платы Versatile Express Cortex-A9 (**vexpress_ca9x4_defconfig**, **CROSS_COMPILE=arm-linux-gnueabi-**). Утилита file подтверждает, что получен исполняемый файл ELF для ARM (рисунок 3). Этот файл QEMU загружает как первый этап через параметр -kernel.

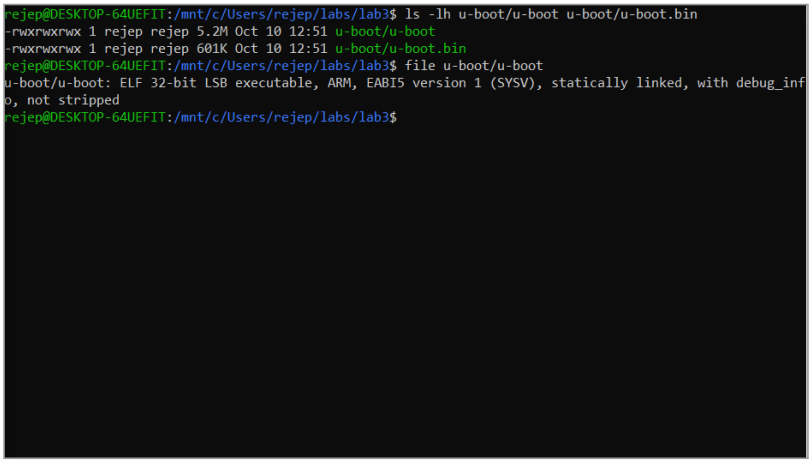


Рисунок 3 — Результат сборки U-Boot: файл u-boot — ELF 32-bit ARM

3.2. Ядро Linux

Ядро сконфигурировано командой **make ARCH=arm vexpress_defconfig** и собрано кросс-компилятором. Получены сжатый образ **zImage** (6,2 МиБ) и дерево устройств **vexpress-v2p-ca9.dtb** (14 КиБ). Для работы схемы initramfs → rootfs в ядре должны быть встроены (значение **y**, не модуль) поддержка initrd/initramfs, devtmpfs, MMC-контроллера ARM MMCI, блочных MMC-устройств и файловой системы ext4. Конфигурация это подтверждает (рисунок 4).

```
rejep@DESKTOP-64UEFIT:/mnt/c/Users/rejep/labs/lab3$ ls -lh linux-7.2.9/arch/arm/boot/zImage linux-7.2.9/
arch/arm/boot/dts/arm/vexpress-v2p-ca9.dtb
-rwxrwxrwx 1 rejep rejep  14K Oct 10 12:53 linux-7.2.9/arch/arm/boot/dts/arm/vexpress-v2p-ca9.dtb
-rwxrwxrwx 1 rejep rejep 6.2M Oct 10 15:56 linux-7.2.9/arch/arm/boot/zImage
rejep@DESKTOP-64UEFIT:/mnt/c/Users/rejep/labs/lab3$ grep -E "CONFIG_(BLK_DEV_INITRD|MMC_ARMMMCI|MMC_BLOC
K|EXT4_FS|DEVTMPFS)=" linux-7.2.9/.config
CONFIG_BLK_DEV_INITRD=y
CONFIG_DEVTMPFS=y
CONFIG_MMC_BLOCK=y
CONFIG_MMC_ARMMMCI=y
CONFIG_EXT4_FS=y
rejep@DESKTOP-64UEFIT:/mnt/c/Users/rejep/labs/lab3$
```

Рисунок 4 — zImage, dtb и ключевые опции ядра в .config

3.3. BusyBox

BusyBox собран со статической линковкой (CONFIG_STATIC=y), поэтому ему не нужны динамические библиотеки и один и тот же бинарник годится и для initramfs, и для rootfs. Результат — статический ARM-исполняемый файл и каталог установки _install (рисунок 5).

```
rejep@DESKTOP-64UEFIT:/mnt/c/Users/rejep/labs/lab3$ file busybox/_install/bin/busybox
busybox/_install/bin/busybox: ELF 32-bit LSB executable, ARM, EABI5 version 1 (SYSV), statically linked,
 BuildID[sha1]=4c80e35e2109aa8ae3e0930bca85dca1b47fface, for GNU/Linux 3.2.0, stripped
rejep@DESKTOP-64UEFIT:/mnt/c/Users/rejep/labs/lab3$ ls busybox/_install
bin  dev  etc  linuxrc  proc  sbin  sys  usr
rejep@DESKTOP-64UEFIT:/mnt/c/Users/rejep/labs/lab3$
```

Рисунок 5 — Статически собранный BusyBox для ARM и каталог _install

4. Initramfs и rootfs

4.1. Initramfs

В каталог initramfs помещены бинарник busybox, узлы устройств /dev/console и /dev/null и скрипт **/init** (рисунок 6). Скрипт выполняет следующее:

- создаёт символические ссылки на апплеты BusyBox и монтирует /proc, /sys и devtmpfs в /dev;
- печатает метку этапа **[STAGE 3] initramfs** и командную строку ядра;
- ждёт появления блочного устройства **/dev/mmcblk0p2** (раздел с rootfs);
- монтирует его как ext4 в /newroot, переносит /dev командой mount -o move, размонтирует /proc и /sys;
- выполняет **exec switch_root /newroot /sbin/init** — корень меняется на настоящий rootfs, а процесс init запускается уже из него.

```
rejep@DESKTOP-64UEFIT:/mnt/c/Users/rejep/labs/lab3$ cat ~/lab3build/initramfs/init
#!/bin/busybox sh
/bin/busybox --install -s /bin
export PATH=/bin:/sbin
mount -t proc none /proc
mount -t sysfs none /sys
mount -t devtmpfs none /dev
echo
echo "=== [STAGE 3] initramfs: /init started (PID $$) ==="
echo "=== [STAGE 3] initramfs: cmdline: $(cat /proc/cmdline)"
echo "=== [STAGE 3] initramfs: waiting for /dev/mmcblk0p2 ..."
i=0
while [ ! -b /dev/mmcblk0p2 ] && [ $i -lt 20 ]; do
  sleep 1; i=$((i+1))
done
ls -l /dev/mmcblk0*
mkdir -p /newroot
if ! mount -t ext4 /dev/mmcblk0p2 /newroot; then
  echo "=== [STAGE 3] initramfs: mount rootfs FAILED, dropping to shell ==="
  exec sh
fi
echo "=== [STAGE 3] initramfs: real rootfs mounted on /newroot, switch_root ... ==="
mount -o move /dev /newroot/dev
umount /proc
umount /sys
exec switch_root /newroot /sbin/init
rejep@DESKTOP-64UEFIT:/mnt/c/Users/rejep/labs/lab3$
```

Рисунок 6 — Скрипт /init, находящийся внутри initramfs

Каталог упаковывается в архив cpio формата newc, сжимается gzip и оборачивается заголовком U-Boot командой mkimage (тип ramdisk). Получается файл **uInitrd** — именно его загружает U-Boot.

4.2. Rootfs

Rootfs — отдельный каталог с файлами системы: содержимое BusyBox _install (bin, sbin, usr), каталоги /proc, /sys, /dev, /tmp, /root, /mnt, а также файлы конфигурации (рисунок 7). **/etc/inittab** говорит init выполнить скрипт /etc/init.d/rcS и запустить оболочку на ttyAMA0. Скрипт **rcS** монтирует /proc, /sys, /tmp и печатает метку **[STAGE 4] rootfs**. Файл **/etc/rootfs-marker** служит наглядным доказательством: его нет в initramfs, он существует только в rootfs. Каталог упакован в образ ext4 командой mke2fs -d.

```
rejep@DESKTOP-64UEFIT:/mnt/c/Users/rejep/labs/lab3$ cat ~/lab3build/rootfs/etc/inittab
::sysinit:/etc/init.d/rcS
ttyAMA0::respawn:-/bin/sh
::ctrlaltdel:/sbin/reboot
::shutdown:/bin/umount -a -r
rejep@DESKTOP-64UEFIT:/mnt/c/Users/rejep/labs/lab3$ cat ~/lab3build/rootfs/etc/init.d/rcS
#!/bin/sh
mount -t proc none /proc
mount -t sysfs none /sys
mount -t tmpfs none /tmp
echo
echo "=== [STAGE 4] rootfs: /sbin/init -> /etc/init.d/rcS started ==="
echo "=== [STAGE 4] rootfs: root device is $(grep ' / ' /proc/mounts | tail -1)"
rejep@DESKTOP-64UEFIT:/mnt/c/Users/rejep/labs/lab3$ cat ~/lab3build/rootfs/etc/rootfs-marker
I am the REAL rootfs (ext4 on /dev/mmcblk0p2), not the initramfs
rejep@DESKTOP-64UEFIT:/mnt/c/Users/rejep/labs/lab3$
```

Рисунок 7 — Файлы rootfs: inittab, rcS и rootfs-marker

5. Сборка образа SD-карты

Все артефакты собираются скриптом build.sh. Ключевые команды:

```
find . | cpio -o -H newc -R 0:0 | gzip -9 > initramfs.cpio.gz
mkimage -A arm -O linux -T ramdisk -C gzip -a 0 -e 0 -n initramfs -d initramfs.cpio.gz uInitrd
mke2fs -t ext4 -L rootfs -d rootfs rootfs.ext4
```

```
mkfs.fat -F 16 -n BOOT boot.fat && mcopy -i boot.fat zImage dtb uInitrd ::
sfdisk sd.img   # p1: FAT 32 MiB (с сектора 2048), p2: Linux 95 MiB (с сектора 67584)
dd if=boot.fat    of=sd.img bs=512 seek=2048  conv=notrunc
dd if=rootfs.ext4 of=sd.img bs=512 seek=67584 conv=notrunc
```

Размер образа 128 МиБ выбран потому, что QEMU требует, чтобы размер SD-карты был степенью двойки. Вывод скрипта (рисунок 8) показывает сборку initramfs и uInitrd, таблицу разделов, содержимое FAT-раздела и проверку ext4 (e2fsck без ошибок).

```
rejep@DESKTOP-64UEFIT:/mnt/c/Users/rejep/labs/lab3$ bash build.sh
[1/5] initramfs
-rw-r--r-- 1 rejep rejep 1.2M Nov  1 14:49 initramfs.cpio.gz
-rw-r--r-- 1 rejep rejep 1.2M Nov  1 14:49 uInitrd
[2/5] rootfs (ext4)
[3/5] FAT-раздел (zImage, dtb, uInitrd)
[4/5] sd.img с таблицей разделов (128 MiB: QEMU требует степень двойки)
[5/5] проверка
Disk /mnt/c/Users/rejep/labs/lab3/sd.img: 128 MiB, 134217728 bytes, 262144 sectors
Units: sectors of 1 * 512 = 512 bytes
Sector size (logical/physical): 512 bytes / 512 bytes
I/O size (minimum/optimal): 512 bytes / 512 bytes
Disklabel type: dos
Disk identifier: 0x104fad68

Device                              Boot Start    End Sectors Size Id Type
/mnt/c/Users/rejep/labs/lab3/sd.img1       2048  67583   65536  32M  c W95 FAT32 (LBA)
/mnt/c/Users/rejep/labs/lab3/sd.img2      67584 262143  194560  95M 83 Linux
--- FAT (p1):
 Volume in drive : is BOOT
 Volume Serial Number is 7534-8506
Directory for ::/

ZIMAGE         6441848 2026-11-01  14:49  zImage
VEXPRE~1 DTB     14329 2026-11-01  14:49  vexpress-v2p-ca9.dtb
UINITRD        1173961 2026-11-01  14:49  uInitrd
        3 files           7 630 138 bytes
                         25 837 568 bytes free

--- ext4 (p2):
e2fsck 1.47.2 (1-Jan-2025)
rootfs: 431/24320 files (0.2% non-contiguous), 3136/24320 blocks

ГОТОВО. Запуск:  bash run.sh      (выход из QEMU: Ctrl+A, потом X)
В U-Boot нажми любую клавишу, чтобы остановить autoboot, и введи:

fatls mmc 0:1
fatload mmc 0:1 0x60000000 zImage
fatload mmc 0:1 0x61000000 vexpress-v2p-ca9.dtb
fatload mmc 0:1 0x62000000 uInitrd
setenv bootargs "console=ttyAMA0 root=/dev/mmcblk0p2 rootwait rdinit=/init"
bootz 0x60000000 0x62000000 0x61000000
rejep@DESKTOP-64UEFIT:/mnt/c/Users/rejep/labs/lab3$
```

Рисунок 8 — Вывод build.sh: initramfs, uInitrd, разделы, проверка

Отдельная проверка образа: таблица разделов и содержимое первого (FAT) раздела (рисунок 9).

```
rejep@DESKTOP-64UEFIT:/mnt/c/Users/rejep/labs/lab3$ sfdisk -l sd.img
Disk sd.img: 128 MiB, 134217728 bytes, 262144 sectors
Units: sectors of 1 * 512 = 512 bytes
Sector size (logical/physical): 512 bytes / 512 bytes
I/O size (minimum/optimal): 512 bytes / 512 bytes
Disklabel type: dos
Disk identifier: 0x104fad68

Device     Boot Start    End Sectors Size Id Type
sd.img1          2048  67583   65536  32M  c W95 FAT32 (LBA)
sd.img2         67584 262143  194560  95M 83 Linux
rejep@DESKTOP-64UEFIT:/mnt/c/Users/rejep/labs/lab3$ mdir -i sd.img@@1M ::
 Volume in drive : is BOOT
 Volume Serial Number is 7534-8506
Directory for ::/

ZIMAGE         6441848 2026-11-01  14:49  zImage
VEXPRE~1 DTB     14329 2026-11-01  14:49  vexpress-v2p-ca9.dtb
UINITRD        1173961 2026-11-01  14:49  uInitrd
        3 files           7 630 138 bytes
                         25 837 568 bytes free

rejep@DESKTOP-64UEFIT:/mnt/c/Users/rejep/labs/lab3$
```

Рисунок 9 — Таблица разделов sd.img и файлы на разделе p1

6. Запуск в QEMU и разбор загрузки по этапам

Система запускается командой:

```
qemu-system-arm -M vexpress-a9 -cpu cortex-a9 -m 512M -kernel u-boot/u-boot \
  -drive file=sd.img,if=sd,format=raw -serial stdio -display none
```

Этап 1. Загрузчик (U-Boot)

QEMU загружает U-Boot, который инициализирует память (DRAM 512 МиБ), MMC-контроллер и последовательный порт. Появляется приглашение «Hit any key to stop autoboot» (рисунок 10); при остановке автозагрузки доступна командная строка U-Boot. Предупреждения pw.conf и pulseaudio в начале вывода относятся к звуковой подсистеме хоста WSL и на загрузку не влияют.

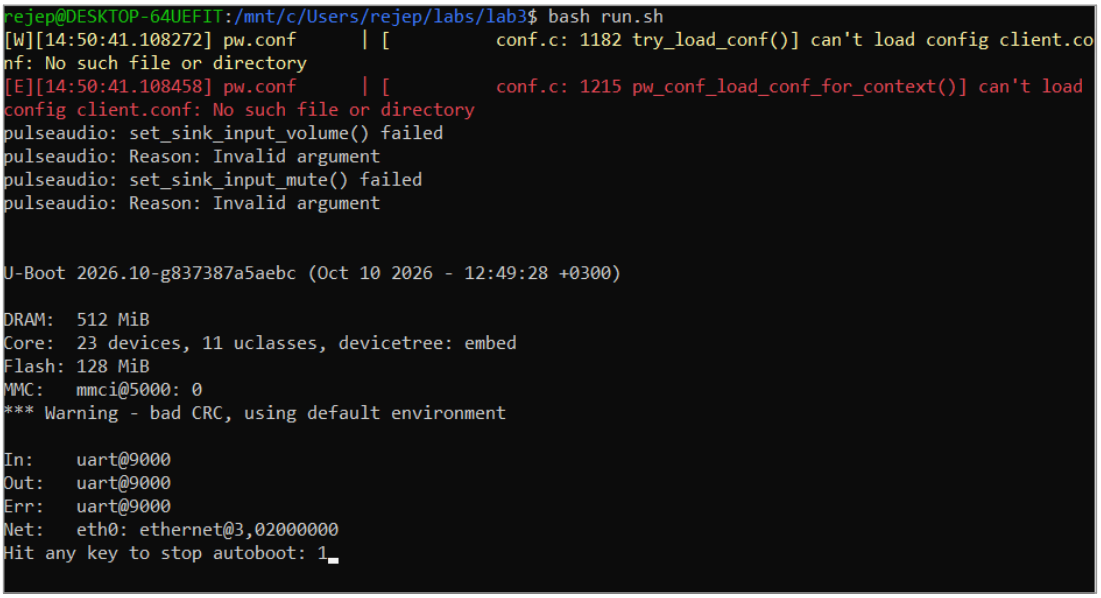


Рисунок 10 — Старт U-Boot в QEMU

Из командной строки U-Boot командой **fatls mmc 0:1** проверено содержимое FAT-раздела, затем три команды **fatload** загружают в память ядро (0x60000000), дерево устройств (0x61000000) и initramfs (0x62000000). В bootargs указаны корневое устройство и **rdinit=/init**, после чего команда **bootz 0x60000000 0x62000000 0x61000000** передаёт управление ядру, передавая ему адреса образа ядра, initramfs и DTB (рисунок 11). U-Boot проверяет контрольную сумму initramfs (Verifying Checksum ... OK) и выводит «Starting kernel ...».

```
=> fatls mmc 0:1
  6441848   zImage
    14329   vexpress-v2p-ca9.dtb
  1173961   uInitrd

3 file(s), 0 dir(s)

=> fatload mmc 0:1 0x60000000 zImage
6441848 bytes read in 9808 ms (640.6 KiB/s)
=> fatload mmc 0:1 0x61000000 vexpress-v2p-ca9.dtb
14329 bytes read in 39 ms (358.4 KiB/s)
=> fatload mmc 0:1 0x62000000 uInitrd
1173961 bytes read in 1812 ms (631.8 KiB/s)
=> setenv bootargs "console=ttyAMA0 root=/dev/mmcblk0p2 rootwait rdinit=/init"
=> bootz 0x60000000 0x62000000 0x61000000
Kernel image @ 0x60000000 [ 0x000000 - 0x624b78 ]
## Loading init Ramdisk from Legacy Image at 62000000 ...
   Image Name:   initramfs
   Image Type:   ARM Linux RAMDisk Image (gzip compressed)
   Data Size:    1173897 Bytes = 1.1 MiB
   Load Address: 00000000
   Entry Point:  00000000
   Verifying Checksum ... OK
## Flattened Device Tree blob at 61000000
   Booting using the fdt blob at 0x61000000
Working FDT set to 61000000
   Loading Ramdisk to 7e9cb000, end 7eae9989 ... OK
   Loading Device Tree to 7e9c4000, end 7e9ca7f8 ... OK
Working FDT set to 7e9c4000

Starting kernel ...
```

Рисунок 11 — Загрузка компонентов в память и запуск ядра командой bootz

Этап 2. Ядро Linux

Ядро стартует (рисунок 12): печатает версию **Linux version 7.2.9**, определяет плату по DTB (Machine model: V2P-CA9) и получает командную строку из bootargs. Далее инициализируются устройства; в логе (рисунок 13) видно обнаружение SD-карты: **mmcblk0: p1 p2** — ядро увидело оба раздела. Сообщения drm-clcd-pl111, ALSA и «deferred probe» относятся к графическому и аудио-оборудованию эмулируемой платы (консоль работает через UART, дисплей отключён) и на загрузку не влияют.

```
   Loading Device Tree to 7e9ed000, end 7e9f37f8 ... OK
Working FDT set to 7e9ed000

Starting kernel ...

Booting Linux on physical CPU 0x0
Linux version 7.2.9 (rejep@DESKTOP-64UEFIT) (arm-linux-gnueabi-gcc (Ubuntu 15.2.0-16ubuntu1) 15.2.0, GNU
 ld (GNU Binutils for Ubuntu) 2.46) #1 SMP Sat Oct 10 15:51:25 MSK 2026
CPU: ARMv7 Processor [410fc090] revision 0 (ARMv7), cr=10c5387d
CPU: PIPT / VIPT nonaliasing data cache, VIPT nonaliasing instruction cache
OF: fdt: Machine model: V2P-CA9
OF: fdt: Ignoring memory block 0x80000000 - 0x80000004
Memory policy: Data cache writeback
cma: Reserved 16 MiB at 0x7f000000
Reserved memory: created DMA memory pool at 0x4c000000, size 8 MiB
OF: reserved mem: initialized node vram@4c000000, compatible id shared-dma-pool
OF: reserved mem: 0x4c000000..0x4c7fffff (8192 KiB) nomap non-reusable vram@4c000000
CPU: All CPU(s) started in SVC mode.
Zone ranges:
  Normal   [mem 0x0000000060000000-0x000000007fffffff]
Movable zone start for each node
Early memory node ranges
  node   0: [mem 0x0000000060000000-0x000000007fffffff]
Initmem setup node 0 [mem 0x0000000060000000-0x000000007fffffff]
```

Рисунок 12 — Начало загрузки ядра: версия 7.2.9, модель платы V2P-CA9

Этапы 3 и 4. Initramfs и переход на rootfs

После инициализации устройств ядро выводит «Run /init as init process» — запускается /init из initramfs (рисунок 13). Скрипт печатает **[STAGE 3] initramfs: /init started (PID 1)**, ждёт устройства и показывает узлы /dev/mmcblk0, mmcblk0p1, mmcblk0p2. Затем раздел монтируется (**EXT4-fs (mmcblk0p2): mounted filesystem**) и выводится **[STAGE 3] initramfs: real rootfs mounted on /newroot, switch_root ...**. После switch_root запускается /sbin/init **уже из rootfs**, что подтверждается строками **[STAGE 4] rootfs: /sbin/init -> /etc/init.d/rcS started** и **[STAGE 4] rootfs: root device is /dev/mmcblk0p2 / ext4 rw**. В конце появляется приглашение оболочки.

```
mmcblk0: mmc0:12d0 QEMU! 128 MiB
aaci-pl041 10004000.aaci: ARM AC'97 Interface PL041 rev0 at 0x10004000, irq 37
aaci-pl041 10004000.aaci: FIFO 512 entries
NET: Registered PF_PACKET protocol family
9pnet: Installing 9P2000 support
Registering SWP/SWPB emulation handler
 mmcblk0: p1 p2
10009000.serial: ttyAMA0 at MMIO 0x10009000 (irq = 38, base_baud = 0) is a PL011 rev1
printk: console [ttyAMA0] enabled
input: ImExPS/2 Generic Explorer Mouse as /devices/platform/bus@40000000/bus@40000000:motherboard-bus@
000000/bus@40000000:motherboard-bus@40000000:iofpga@7,00000000/10007000.kmi/serio1/input/input2
1000a000.serial: ttyAMA1 at MMIO 0x1000a000 (irq = 39, base_baud = 0) is a PL011 rev1
1000b000.serial: ttyAMA2 at MMIO 0x1000b000 (irq = 40, base_baud = 0) is a PL011 rev1
1000c000.serial: ttyAMA3 at MMIO 0x1000c000 (irq = 41, base_baud = 0) is a PL011 rev1
drm-clcd-pl111 1001f000.clcd: assigned reserved memory node vram@4c000000
drm-clcd-pl111 1001f000.clcd: [drm] using device-specific reserved memory
drm-clcd-pl111 1001f000.clcd: [drm] core tile graphics present
drm-clcd-pl111 1001f000.clcd: [drm] this device will be deactivated
drm-clcd-pl111 1001f000.clcd: [drm] *ERROR* Versatile Express init failed - -19
drm-clcd-pl111 10020000.clcd: [drm] DVI muxed to daughterboard 1 (core tile) CLCD
drm-clcd-pl111 10020000.clcd: [drm] initializing Versatile Express PL111
drm-clcd-pl111 10020000.clcd: [drm] DVI muxed to daughterboard 1 (core tile) CLCD
drm-clcd-pl111 10020000.clcd: [drm] initializing Versatile Express PL111
drm-clcd-pl111 10020000.clcd: [drm] DVI muxed to daughterboard 1 (core tile) CLCD
drm-clcd-pl111 10020000.clcd: [drm] initializing Versatile Express PL111
clk: Disabling unused clocks
ALSA device list:
  #0: ARM AC'97 Interface PL041 rev0 at 0x10004000, irq 37
Freeing unused kernel image (initmem) memory: 1024K
Run /init as init process

=== [STAGE 3] initramfs: /init started (PID 1) ===
=== [STAGE 3] initramfs: cmdline: console=ttyAMA0 root=/dev/mmcblk0p2 rootwait rdinit=/init
=== [STAGE 3] initramfs: waiting for /dev/mmcblk0p2 ...
brw-------    1 0        0         179,   0 Nov  1 11:57 /dev/mmcblk0
brw-------    1 0        0         179,   1 Nov  1 11:57 /dev/mmcblk0p1
brw-------    1 0        0         179,   2 Nov  1 11:57 /dev/mmcblk0p2
EXT4-fs (mmcblk0p2): recovery complete
EXT4-fs (mmcblk0p2): mounted filesystem 4cb08439-8cff-48a8-bb30-b6d4379dee55 r/w with ordered data mod
 Quota mode: disabled.
=== [STAGE 3] initramfs: real rootfs mounted on /newroot, switch_root ... ===
random: crng init done

=== [STAGE 4] rootfs: /sbin/init -> /etc/init.d/rcS started ===
=== [STAGE 4] rootfs: root device is /dev/mmcblk0p2 / ext4 rw,relatime 0 0
~ # drm-clcd-pl111 10020000.clcd: [drm] DVI muxed to daughterboard 1 (core tile) CLCD
drm-clcd-pl111 10020000.clcd: [drm] initializing Versatile Express PL111
i2c 0-0039: deferred probe pending: sii902x: Failed to find remote bridge
amba 1000f000.wdt: deferred probe pending: (reason unknown)
```

Рисунок 13 — Логи: Run /init, этап initramfs, монтирование ext4, switch_root, этап rootfs

Журнал ядра (dmesg) повторно подтверждает порядок событий (рисунок 14): версия ядра и командная строка, «Unpacking initramfs...», «Freeing initrd memory», обнаружение разделов mmcblk0 p1 p2, «Run /init as init process» и только после этого — монтирование EXT4 с mmcblk0p2.

```
~ # dmesg | grep -iE "Linux version|command line|initramfs|initrd|Run /init|mmcb
lk|EXT4"
Linux version 7.2.9 (rejep@DESKTOP-64UEFIT) (arm-linux-gnueabi-gcc (Ubuntu 15.2.0-16ubuntu1) 15.2.0, GNU
 ld (GNU Binutils for Ubuntu) 2.46) #1 SMP Sat Oct 10 15:51:25 MSK 2026
Kernel command line: console=ttyAMA0 root=/dev/mmcblk0p2 rootwait rdinit=/init
Unpacking initramfs...
Freeing initrd memory: 1148K
mmcblk0: mmc0:cf86 QEMU! 128 MiB
 mmcblk0: p1 p2
Run /init as init process
EXT4-fs (mmcblk0p2): mounted filesystem 4cb08439-8cff-48a8-bb30-b6d4379dee55 r/w with ordered data mode.
 Quota mode: disabled.
```

Рисунок 14 — dmesg: распаковка initramfs, запуск /init, монтирование ext4

7. Подтверждение: корень системы — rootfs на ext4

В запущенной системе проверено (рисунок 15):

- **mount | grep " / "** показывает **/dev/mmcblk0p2 on / type ext4** — корнем является раздел SD-карты, а не RAM-диск initramfs (в схеме без switch_root здесь было бы «rootfs on / type rootfs»);
- **cat /etc/rootfs-marker** выводит «I am the REAL rootfs ...» — файл существует только в rootfs;
- **cat /proc/cmdline** содержит rdinit=/init и root=/dev/mmcblk0p2;
- **uname -a** показывает Linux 7.2.9 armv7l;
- в корне присутствует каталог **lost+found**, характерный для ext4.

```
~ # mount | grep " / "
/dev/mmcblk0p2 on / type ext4 (rw,relatime)
~ # cat /etc/rootfs-marker
I am the REAL rootfs (ext4 on /dev/mmcblk0p2), not the initramfs
~ # cat /proc/cmdline
console=ttyAMA0 root=/dev/mmcblk0p2 rootwait rdinit=/init
~ # uname -a
Linux (none) 7.2.9 #1 SMP Sat Oct 10 15:51:25 MSK 2026 armv7l GNU/Linux
~ # ls -l /
total 40
drwxr-xr-x    2 0        0            4096 Oct 10 13:46 bin
drwxr-xr-x    4 0        0            2780 Nov  1 11:51 dev
drwxr-xr-x    3 0        0            4096 Nov  1 11:49 etc
lrwxrwxrwx    1 0        0              11 Oct 10 13:46 linuxrc -> bin/busybox
drwx------    2 0        0           16384 Nov  1 11:49 lost+found
drwxr-xr-x    2 0        0            4096 Nov  1 11:49 mnt
dr-xr-xr-x   75 0        0               0 Nov  1 11:51 proc
drwxr-xr-x    2 0        0            4096 Nov  1 11:49 root
drwxr-xr-x    2 0        0            4096 Nov  1 11:49 sbin
dr-xr-xr-x   12 0        0               0 Nov  1 11:51 sys
drwxrwxrwt    2 0        0              40 Nov  1 11:51 tmp
drwxr-xr-x    4 0        0            4096 Oct 10 13:46 usr
~ #
```

Рисунок 15 — Проверка в запущенной системе: mount, rootfs-marker, cmdline, uname, ls -l /

8. Сводная таблица: этап загрузки и подтверждающая строка лога

Этап	Что происходит	Подтверждающие строки

1. Загрузчик (U-Boot)	Инициализация железа, чтение файлов с FAT-раздела p1, передача управления ядру	`U-Boot 2026.10 ...`
		`fatload mmc 0:1 0x60000000 zImage`
		`Verifying Checksum ... OK`
		`Starting kernel ...`

2. Ядро	Запуск ядра, разбор DTB, обнаружение SD-карты и её разделов	`Linux version 7.2.9 ...`
		`Kernel command line: console=ttyAMA0 root=/dev/mmcblk0p2 rootwait rdinit=/init`
		`mmcblk0: p1 p2`

3. initramfs	Распаковка архива в память, запуск /init из него	`Unpacking initramfs...`
		`Run /init as init process`
		`=== [STAGE 3] initramfs: /init started (PID 1) ===`

переход (switch_root)	Монтирование ext4-раздела и смена корня	`EXT4-fs (mmcblk0p2): mounted filesystem` `=== [STAGE 3] initramfs: real rootfs mounted on /newroot, switch_root ... ===`

4. rootfs	Запуск /sbin/init и rcS уже с раздела ext4, оболочка	`=== [STAGE 4] rootfs: /sbin/init -> /etc/init.d/rcS started ===`
		`mount: /dev/mmcblk0p2 on / type ext4`
		`cat /etc/rootfs-marker`

9. Вывод

В ходе работы из исходных кодов собраны загрузчик U-Boot, ядро Linux 7.2.9 и BusyBox, подготовлены отдельные initramfs (uInitrd) и rootfs (образ ext4), которые размещены на двух разделах образа SD-карты. В QEMU на плате vexpress-a9 получена полная цепочка загрузки **U-Boot → ядро (zImage + DTB) → initramfs → switch_root → rootfs**. Каждый этап подтверждён логом: U-Boot загружает ядро и initramfs, ядро распаковывает initramfs и запускает /init, /init монтирует ext4-раздел и выполняет switch_root, после чего система работает с корневой файловой системой /dev/mmcblk0p2 (type ext4). Тем самым требование задания о порядке загрузки выполнено полностью, включая этап initramfs.